



POLITECNICO
MILANO 1863

Intelligent Job Management – Jan Wangrat

Intelligent Job Management

Final Report

Author: Jan Wangrat

Supervisor: Prof. Danilo Ardagna, Ph.D. Candidate Federica Filippini

Date: May 13, 2026

Deliverable:	Final Report
Title:	Intelligent Job Management
Author:	Jan Wangrat
Supervisor:	Prof. Danilo Ardagna, Ph.D. Candidate Federica Filippini
Version:	1.0
Date:	May 13, 2026

Contents

Table of Contents	3
List of Figures	4
List of Tables	4
1 Introduction	6
2 System Architecture	7
2.1 Deployment topology	7
2.2 Scheduling pipeline	9
2.3 State machine	13
2.4 Concurrency invariants	13
2.5 Cross-node checkpoints and image variants	14
3 Configuration & Job Submission	15
3.1 Cluster Configuration	15
3.1.1 Node Configuration (nodes_config.json)	15
3.1.2 GPU Energy Costs (gpu_energy_costs.json)	16
3.2 Job Submission	16
3.2.1 Request Body	16
3.2.2 Response (201 Created)	17
3.3 Full API Overview	17
3.4 Runtime environment	18
4 End-to-End Scenarios	19
4.1 Objective and cost model	19
4.2 Two-GPU benefit — where it actually appears	19
4.3 Scenario 1 — single type	20
4.4 Scenario 2 — two types	21
4.5 Reported numbers and where they live	24
References	25

List of Figures

1	Distributed prototype: SPA + API + optimizer on the operator’s machine, workers + Docker on each GPU node, PostgreSQL on a GPU node, all glued together over SSH-tunneled HTTP. Checkpoints written by one node are visible from the other via a shared rclone-over-SFTP mount.	8
2	Submission and scheduling. The API only writes the row and notifies the scheduler; placement is decided by the next scheduling pass. The optimizer call is made outside the schedule lock so its 30 s HTTP timeout cannot block other passes; stale optimizer suggestions are safely filtered by Phase 1b’s conditional update. Slot semaphores ensure that no more containers than GPUs are dispatched to any worker.	10
3	Profiling cycle.	11
4	Optimizer cycle.	12
5	Standard run execution. The worker performs an atomic conditional UPDATE . . . WHERE status = ANY(RUNNABLE_STATUSES) so concurrent stops / duplicate dispatches cannot race a row into RUNNING. On exit the worker fires both <code>ijm_slot_freed</code> (which releases the API’s NodeSlots permit) and <code>ijm_schedule</code> (which wakes the scheduler) in the same transaction.	13
6	Scenario 1 timeline. Stages: S1 sweep+fill, S2 URGENT auto-preempt, S3 user-stop, S4 cross-node resume. Bar colour encodes the node (blue = <code>polimi-gpu/A40</code> , red = <code>matemagician/P600</code>); the <code>x1/x2</code> label gives the GPU count; hatched fill marks profile runs. S2 evicts JOB2 from its A40 slot; the P600 slot freed by S3’s user-stop on JOB3 carries JOB2’s cross-node resume at S4.	22
7	Scenario 2 timeline. Stages: S1 sweep+fill, S2 URG-S preempts, S3 URG-B preempts, S4.5 user-stop + JOB7 backfill. Same visual grammar as Fig. 6. Both <code>prio=1</code> A40 occupants are evicted (S2 + S3) and resume on <code>matemagician</code> (two cross-node checkpoint loads); at S4.5 JOB7 (opposite type) backfills the P600 slot freed by the user-stop on JOB4.	24

List of Tables

1	Node configuration fields.	15
2	Job creation request fields.	16
3	API endpoints.	17
4	Key environment variables.	18
5	Left: hourly rates fed to the optimiser (<code>config/gpu_energy_costs.json</code>). Two-GPU bundles are <i>not</i> exactly twice the one-GPU rate (small co-location discount, 0.55 vs 0.60 for A40×2 and 0.055 vs 0.060 for P600×2). Right: tardiness weight by priority (derived from urgency scaling).	19
6	Measured per-epoch times from the profile sweep for <code>cnn_big</code> (10 conv blocks, 128×128×3 input, batch 32). Both GPU classes now show a 2-GPU wall-clock speedup: A40×2 is 11% faster than A40×1, and P600×2 is 68% faster than P600×1 (1.68× speedup) — DataParallel delivers near-linear scaling on P600 because each step’s forward+backward pass is large enough to fully amortise the gradient sync. On A40 the per-step compute is smaller but still big enough that the 2-GPU split clears the NCCL overhead.	20
7	Per-bundle ExecutionTime and GPUcost the optimiser sees for every <code>lstm-small</code> instance at the S1 round. Profile coverage is 4 rows (all four cells) for <code>lstm-small</code> . All deadlines $\geq 1\text{ h} \times$ the longest 40-epoch run, so TardCost = 0 across every plan and the cost ranking is pure GPUcost.	20

8	Optimiser plan comparison for the URGENT submission, computed from the S2 round CSV. Choosing A40 (and evicting whichever A40 occupant has the smallest $\text{priority} \times \text{remaining-runtime}$ score — observed JOB2) saves \$0.086 over leaving URGENT to queue on P600.	21
9	Eight rows of measured profile data (four bundles, two types). For <code>cnn_big</code> , <i>both</i> 2-GPU bundles are wall-clock-faster than their 1-GPU sibling: A40×2 is 11% faster (5.63 vs 6.29 s/epoch) and P600×2 is 68% faster (30.32 vs 51.06 s/epoch). On cost terms P600×1 remains the cost-minimum for patient jobs (\$0.0170 vs P600×2’s \$0.0185, $\Delta = \$0.0015$), but the 9% premium is tiny enough that any deadline pressure flips the decision in P600×2’s favour. A40×2 stays cost-dominated by A40×1 because the 11% saving doesn’t pay for the 64% hourly-rate hike. The LSTM rows show the opposite regime — DataParallel sync time already dominates the per-step compute, so 2-GPU never wins meaningfully there.	22
10	Optimiser plan comparison for the two URGENT submissions under the <code>cnn_big</code> “B” type. URG-S still goes to the cheap P600 (its 219 s ExecTime keeps tardiness small). URG-B picks A40×1, but A40×2 is now a real 2nd-place contender at \$0.0943 (only 1.5% behind) — if even one A40 slot were unevictable the optimiser would flip to A40×2 and evict the two cheaper-to-displace A40 occupants together. Lower down the ranking, QuadroP600×2 (\$0.517) beats QuadroP600×1 (\$0.884) by 1.7× TotalCost purely because the 1.68× wall-clock speedup of DataParallel on <code>cnn_big</code> nearly halves the tardiness penalty. Both 2-GPU plans are real candidates the optimiser <i>ranks</i> , not just profile-sweep theatre.	23

1 Introduction

2 System Architecture

2.1 Deployment topology

IJM runs as a small distributed system. The FastAPI *API* (which also hosts the *ProfilingScheduler*, *JobDispatcher*, *NodeSlots* semaphores and the optimizer client) runs on the operator's machine. A lightweight FastAPI *worker* runs on every GPU node and owns the local Docker daemon. PostgreSQL runs on one of the GPU nodes. Communication is plain HTTP over SSH tunnels, so the API reaches each worker at a stable `http://localhost:800x` alias regardless of where it actually sits. An optional GPUspb *optimizer* HTTP service is reached from the API.

Current Prototype Architecture (distributed)

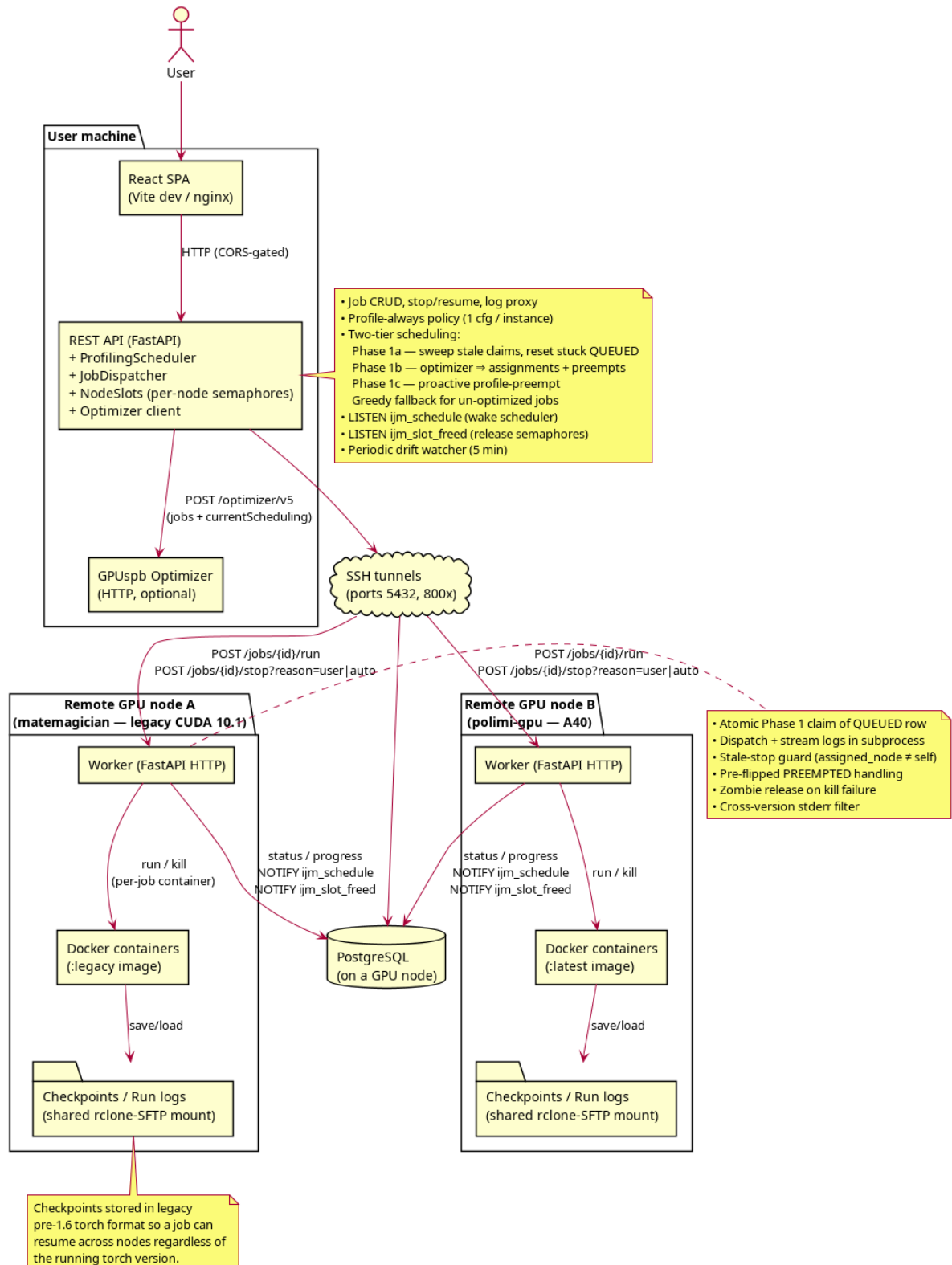


Figure 1: Distributed prototype: SPA + API + optimizer on the operator’s machine, workers + Docker on each GPU node, PostgreSQL on a GPU node, all glued together over SSH-tunneled HTTP. Checkpoints written by one node are visible from the other via a shared rclone-over-SFTP mount.

Single-process docker compose on the operator’s machine is supported for development but is not

the deployment target: any tradeoff between local-dev simplicity and distributed correctness resolves toward the distributed topology.

2.2 Scheduling pipeline

A single scheduling pass — `_schedule_waiting_jobs` in `app.py` — handles every dispatch decision. It is invoked whenever a job is submitted, a profiling run finishes, a slot is freed, or every 60 s by a watchdog. Notices are coalesced through a single `asyncio.Event` and rate-limited to at most one pass every five seconds so the cluster has time to apply the previous plan before the next one is computed.

The pass walks four phases under a single `schedule_lock`:

Phase 0 Sweep stale profile claims (rows in `profiling_results` with `duration_seconds` IS NULL whose owning instance no longer exists or was repurposed) and reset stuck QUEUED rows whose `assigned_node` was set ≥ 30 s ago without the worker advancing them.

Phase 1b Call the optimizer (outside the lock for its 30 s HTTP timeout) and apply its assignments via conditional UPDATE ...WHERE `assigned_node` IS NULL; collect its preempt list.

Greedy fallback For QUEUED rows the optimizer did not place (typically rows still owing a profile run), the `ProfilingScheduler` picks one profile config or a profiled standard slot.

Phase 1c If a row still has no slot and still owes a profile, find the cheapest set of low-priority RUNNING jobs whose eviction would free enough GPUs for the next unprofiled config (the *profile-always* policy).

Outside the lock the pass spawns one task per preempt and one per dispatch. Each dispatch goes through `JobDispatcher.dispatch_with_slot`, which acquires the destination's `NodeSlots` permit before sending POST `/jobs/{id}/run` to the right worker. Permits are released when the worker emits a NOTIFY `ijm_slot_freed` carrying the freed (`node_id`, `n_gpus`) pair.

Figure 2 shows the submission path, Figure 3 the cross-instance profile cycle, Figure 4 an optimizer-driven preempt+migrate round, and Figure 5 the standard run.

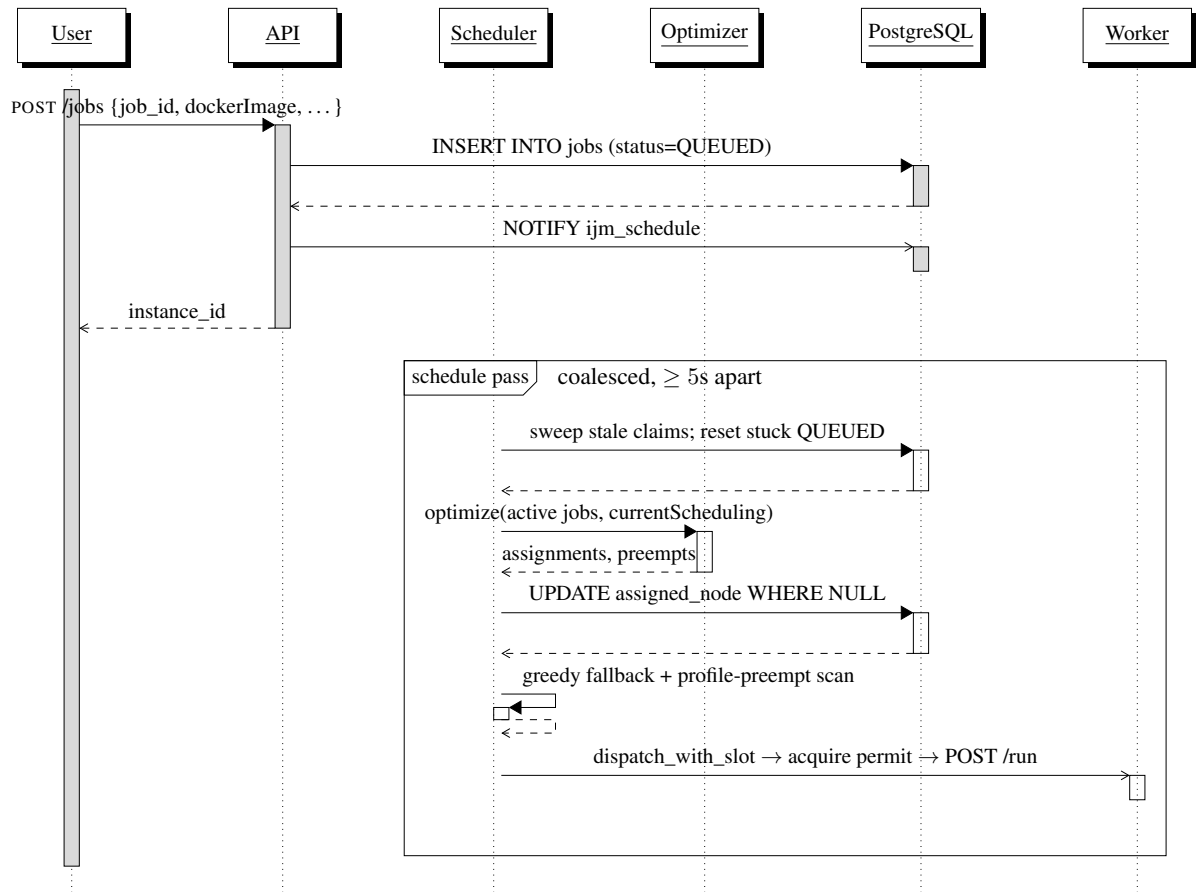


Figure 2: Submission and scheduling. The API only writes the row and notifies the scheduler; placement is decided by the next scheduling pass. The optimizer call is made outside the schedule lock so its 30 s HTTP timeout cannot block other passes; stale optimizer suggestions are safely filtered by Phase 1b’s conditional update. Slot semaphores ensure that no more containers than GPUs are dispatched to any worker.

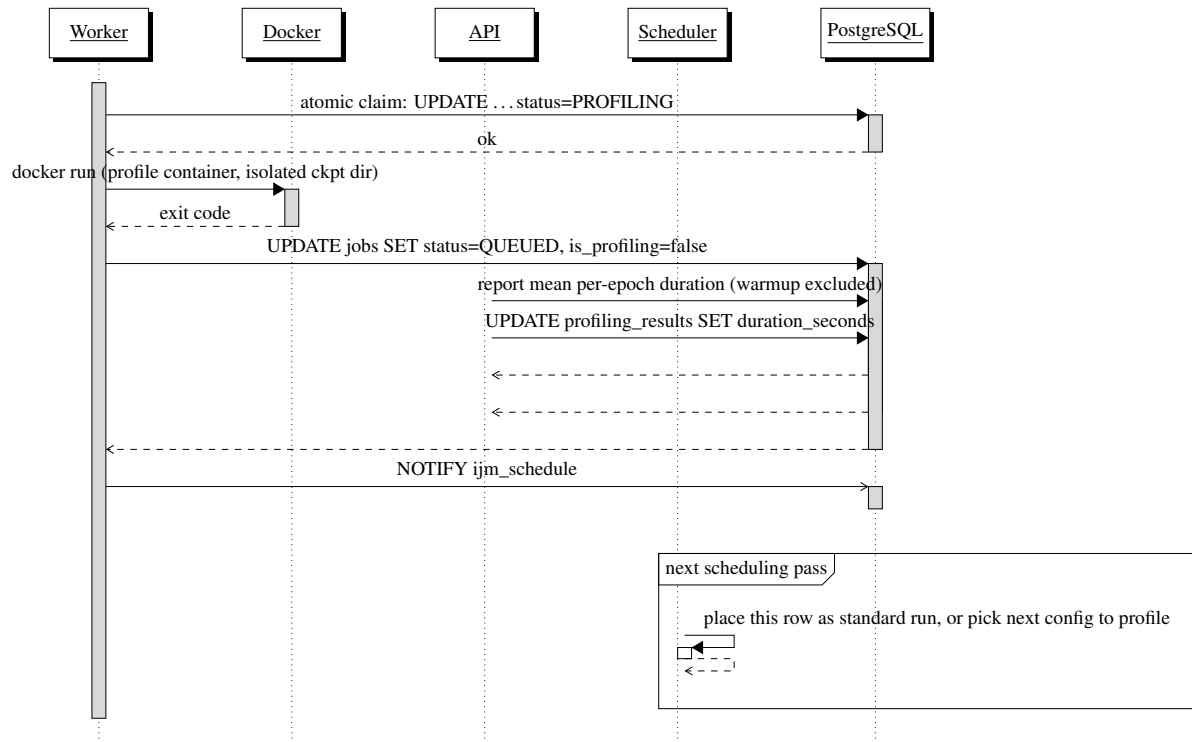


Figure 3: Profiling cycle. Profiling results are keyed by the (job-type, gpu-config) pair and shared across submissions of the same type. Each instance contributes one profile run by default (PROFILING_CONFIGS_PER_JOB); the per-instance claim row guards against duplicate measurements. Mean per-epoch time is computed from monotonic timestamps with the first (warmup) epoch excluded.

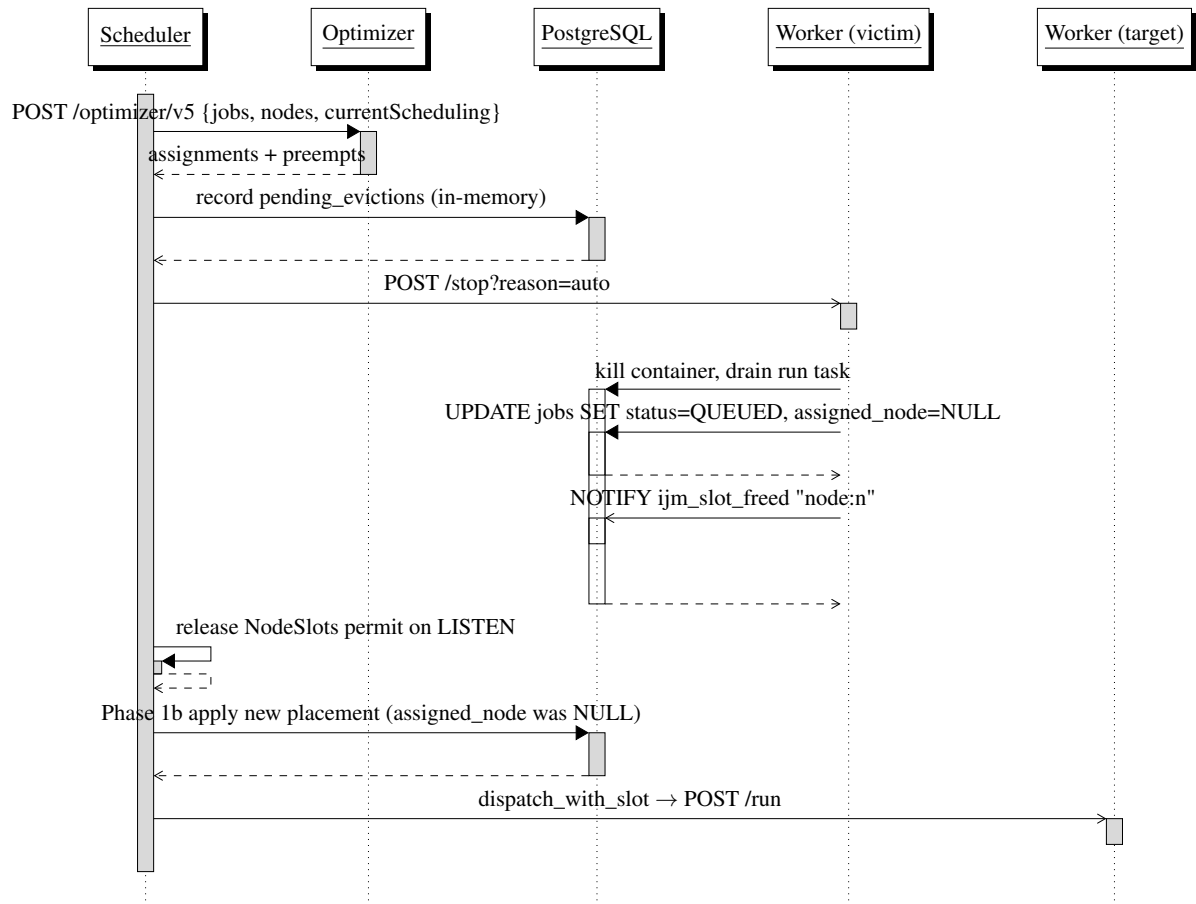


Figure 4: Optimizer-driven preempt and migrate. The API trusts GPUspb’s plan: any job in `currentScheduling` that the optimizer drops or moves to a different slot is preempted with `reason=auto`, which clears its assignment so Phase 1b can re-apply the new placement. An in-memory pending-evictions map makes freshly-evicted slots visible to the scheduler before the worker’s slot-freed commit lands, so multi-victim profile preempts do not deadlock.

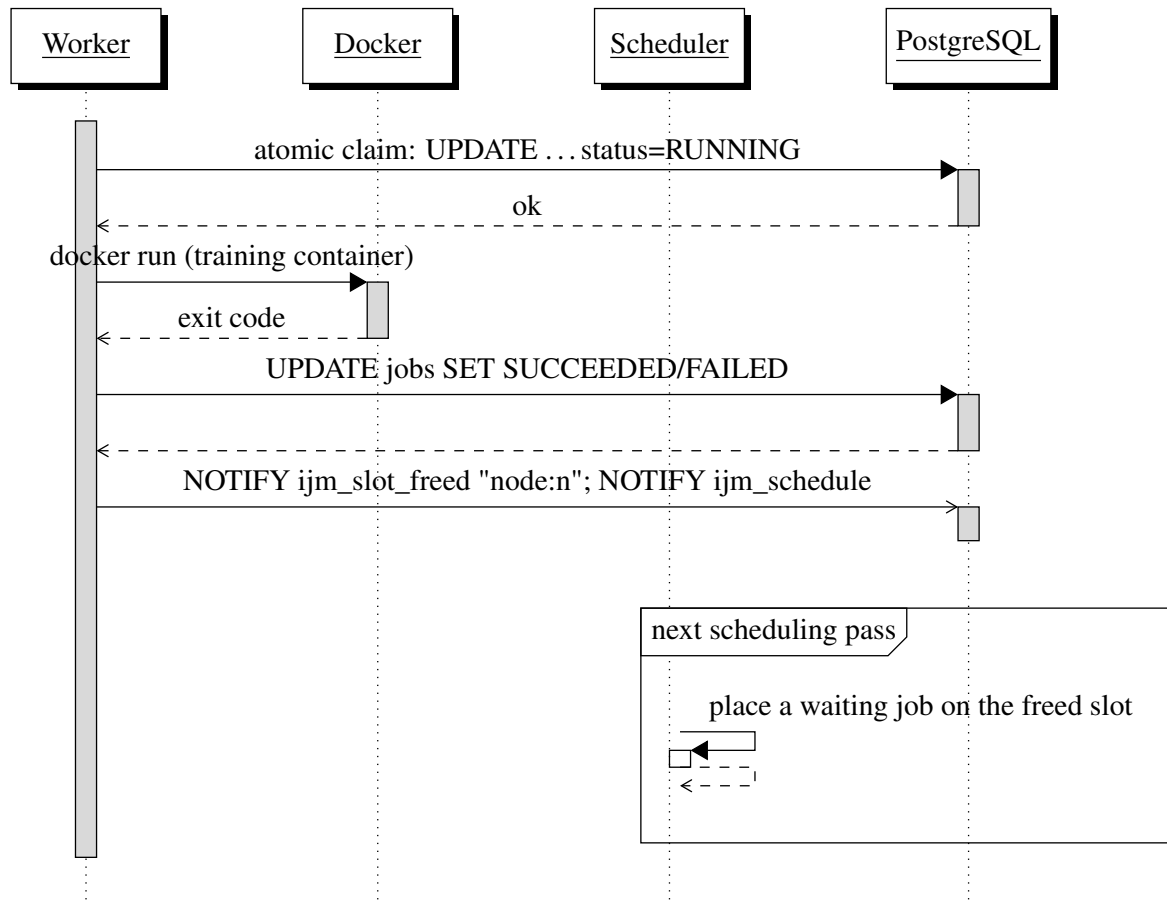


Figure 5: Standard run execution. The worker performs an atomic conditional `UPDATE ...WHERE status = ANY(RUNNABLE_STATUSES)` so concurrent stops / duplicate dispatches cannot race a row into `RUNNING`. On exit the worker fires both `ijm_slot_freed` (which releases the API's NodeSlots permit) and `ijm_schedule` (which wakes the scheduler) in the same transaction.

2.3 State machine

A job moves through the following states. All transitions are driven by atomic conditional `UPDATES` so concurrent stop / dispatch / completion paths cannot clobber each other's work.

From	To	Trigger
QUEUED	PROFILING / RUNNING	Worker Phase 1 atomic claim
PROFILING	QUEUED	Profile run finished; <code>is_profiling_run</code> flips to false
RUNNING	SUCCEEDED / FAILED	Container exit
RUNNING	PREEMPTED	User POST <code>/jobs/{id}/stop?reason=user</code>
RUNNING	QUEUED	Auto-preempt <code>reason=auto</code> : clears <code>assigned_node</code> so the next pass can place elsewhere
PREEMPTED / FAILED	QUEUED	POST <code>/jobs/{id}/resume</code>

Two distinct stop reasons separate *user-initiated* preemption (sticky `PREEMPTED`, the user must explicitly resume) from *scheduler-initiated* preemption (`QUEUED` with cleared assignment, picked up automatically on the next scheduling pass). The worker enforces this distinction and is also responsible for ignoring stale `/stop` calls aimed at a row that has since been re-assigned to a different node.

2.4 Concurrency invariants

The control plane relies on three invariants, each defended in code:

- **No GPU oversubscription.** Every dispatch goes through `NodeSlots.acquire(node_id, n_gpus)`. On startup the semaphores are reconciled from the DB (rows in `RUNNING/PROFILING` or `QUEUED` with an assigned node pre-acquire their permits). A periodic drift watcher recomputes the authoritative count every five minutes and corrects any divergence.
- **No duplicate containers per row.** Both the worker (Phase 1 claim, `running_jobs[job_id]` check) and the API (`dispatch_tasks[instance_id]` deduplication) refuse a second concurrent dispatch for the same instance. The reaper that resets stuck `QUEUED` rows cancels any live dispatch task instead of double-releasing its permit.
- **Profiling is sacred.** `Auto-preempt (_preempt_and_release)` refuses to act on `is_profiling_run=TRUE`; `profile-preempt (Phase 1c)` filters victims by `status='RUNNING'` only. Only an explicit user `/stop?reason=user` can cancel an in-flight measurement.

2.5 Cross-node checkpoints and image variants

Two GPU generations are supported. The `matemagician` node is pinned to driver 418.x (CUDA 10.1 max), so it runs a separate `:legacy` image built on PyTorch 1.5.1 + Python 3.7. Newer nodes run the default `:latest` image on PyTorch 2.6 + CUDA 12.4. Checkpoints are saved in the legacy pre-1.6 torch serialization format on every node so that a job can resume across nodes regardless of which torch version produced the file.

The shared `data/` directory is mounted on every node via `rclone` over SFTP and contains:

- `data/checkpoints/{job_id}/` — per-job checkpoints, mounted into the container at `/checkpoints`;
- `data/runs/{job_id}/` — captured stdout (`output.log`) and TensorBoard artifacts, mounted at `/runs`;
- `data/shared/data/` — read-only dataset cache (MNIST, CIFAR-10), mounted at `/runs/data`.

Worker-side environment variables `WORKER_GPU_MODE` (`runtime` | `cdi` | `none`) and `IMAGE_TAG_OVERRIDE` let the same `nodes_config` drive heterogeneous nodes: the `polimi-gpu` rootless daemon uses CDI device injection, `matemagician` rewrites `:latest` to `:legacy`, and a CPU-only sandbox can be added by setting `WORKER_GPU_MODE=none`.

3 Configuration & Job Submission

This section describes the system configuration files and the job submission API.

3.1 Cluster Configuration

Cluster topology is defined in two JSON files loaded at startup (paths configurable via environment variables).

3.1.1 Node Configuration (nodes_config.json)

Each entry describes a compute node with its GPU resources.

```
[
  {
    "id": "matemagician",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8001",
    "cost": 0.06,
    "resources": [
      { "gpu_type": "QuadroP600", "gpu_count": 2 }
    ]
  },
  {
    "id": "polimi-gpu",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8002",
    "cost": 0.55,
    "resources": [
      { "gpu_type": "A40", "gpu_count": 2 }
    ]
  }
]
```

Field	Type	Description
id	string	Unique node identifier
isForProfiling	bool	If true, the node can host profiling runs. Production jobs may still land on it; profiling jobs may only land on it
workerUrl	string null	HTTP base URL of the worker on this node, typically reached through an SSH tunnel. When null, dispatch falls through to the in-process JobRunner (only useful for the docker-compose all-in-one development setup)
cost	float	Idle cost of the node (\$/h) used by the optimizer
resources	array	GPU resources attached to the node
resources[].gpu_type	string	GPU model name (e.g. A40, L40S, QuadroP600)
resources[].gpu_count	int	Number of GPUs of this type

Table 1: Node configuration fields.

GPU type and count are declared explicitly in the configuration rather than auto-detected at runtime. This allows mixed-GPU nodes (e.g. a node with both A40 and L40S GPUs), which ANDREAS did not support, and lets the operator declare nodes that aren't physically attached to the API host — the API only ever talks to workers over HTTP.

3.1.2 GPU Energy Costs (gpu_energy_costs.json)

Maps GPU type and count to hourly energy cost (EUR/h), used by the scheduler.

```
{
  "A40": { "1": 0.15, "2": 0.28, "4": 0.54 },
  "L40S": { "1": 0.18, "2": 0.34, "4": 0.65 }
}
```

3.2 Job Submission

Endpoint: POST /jobs

3.2.1 Request Body

```
{
  "job_id": "LSTM-small",
  "dockerImage": "ijm-lstm-small:dev",
  "scriptPath": "train.py",
  "directoryToMount": "/data/my-project",
  "Priority": 3,
  "deadline": "2026-03-20T18:00:00",
  "batchSize": 64,
  "profilingEpochsNo": 3,
  "epochsTotal": 20
}
```

Field	Type	Required	Description
job_id	string	yes	Reusable job type identifier; profiling results are correlated across submissions sharing the same job_id
dockerImage	string	yes	Docker image to execute
scriptPath	string	no	Path to training script (overrides Docker CMD with python3 <scriptPath>)
directoryToMount	string	no	Host directory mounted into the container at /workspace
command	string[]	no	Command and arguments (defaults to image entry-point; overridden by scriptPath)
Priority	int 1–5	no	Job priority (default 3)
deadline	datetime	no	Job deadline (ISO 8601)
batchSize	int	no	Batch size passed as BATCH_SIZE env var
profilingEpochsNo	int ≥ 1	no	Epochs for profiling runs (default 3)
epochsTotal	int ≥ 1	no	Total training epochs (default 20)

Table 2: Job creation request fields.

Each submission receives a unique UUID instance ID (id) for lifecycle tracking, separate from the job_id type identifier. When scriptPath is provided, the container runs python3 <scriptPath>; otherwise the Docker image’s default CMD is used. Unlike ANDREAS, Priority and deadline are optional, and an additional command field provides flexibility for non-standard workloads.

3.2.2 Response (201 Created)

Returns the full job object including scheduler decisions:

```
{
  "id": "a1b2c3d4-...",
  "job_id": "LSTM-small",
  "image": "ijm-lstm-small:dev",
  "command": ["python3", "train.py"],
  "status": "QUEUED",
  "is_profiling_run": true,
  "assigned_node": "node-mixed-prof",
  "assigned_gpu_config": { "A40": 1 },
  "priority": 3,
  "epochs_total": 20,
  "profiling_epochs_no": 3,
  "created_at": "2026-03-18T10:00:00Z",
  "updated_at": "2026-03-18T10:00:00Z",
  ...
}
```

The response immediately shows the scheduling decision: assigned node, GPU configuration, and whether this is a profiling run.

3.3 Full API Overview

Table 3 lists all endpoints. Interactive documentation is available at `/docs` (Swagger UI) and `/redoc` when the API is running.

Method	Path	Description
GET	/	Health check (basic)
GET	/health	Detailed health (DB + job runner)
GET	/nodes	List cluster nodes with status
GET	/gpu-costs	GPU energy cost table
GET	/configurations	Valid GPU configurations
POST	/jobs	Submit a new job
GET	/jobs	List all jobs (paginated)
GET	/jobs/{id}	Get job by ID
DELETE	/jobs/{id}	Delete a job
DELETE	/jobs	Delete all jobs
POST	/jobs/{id}/stop	Stop a running job
POST	/jobs/{id}/resume	Resume a stopped/failed job
GET	/jobs/{id}/logs	Retrieve job output log
GET	/profiling-results/{id}	Profiling results for a job type

Table 3: API endpoints.

All scheduling, profiling, and dispatch decisions are consolidated into a single FastAPI service, in contrast to ANDREAS which split these across a Java Jobs Manager and a C++ Jobs Optimizer. The actual execution lives in per-node *workers* (also FastAPI), which the API reaches over HTTP through SSH tunnels. An optional external GPUspb optimizer is reached the same way: when its URL is unset the system falls back to the in-API greedy scheduler. IJM exposes explicit `stop`, `resume`, `logs`, and

delete endpoints, as well as `/profiling-results` and `/configurations` to give users visibility into the scheduler’s state.

3.4 Runtime environment

The most important runtime knobs (all set in the API process unless noted otherwise) are:

Variable	Default	Description
DATABASE_URL	postgresql://.../ijm	PostgreSQL DSN, reached via SSH tunnel
OPTIMIZER_URL	unset	When set, the API delegates placement to the GPUspb HTTP service. Unset $\Rightarrow$ greedy scheduler only
OPTIMIZER_METHOD	RG	GPUspb solver method passed in the request body
OPTIMIZER_VERBOSE	0	GPUspb per-call log verbosity (0–3)
OPTIMIZER_SWITCH_PENALTY_S	60	Wall-time penalty (s) charged when judging optimizer migrations without destination profiling data
IJM_OPTIMIZER_GUARD_MIGRATIONS	unset	When set, re-enables the local cost-benefit filter on optimizer-proposed migrations (default trusts GPUspb)
PROFILING_CONFIGS_PER_JOB	1	How many distinct GPU configs a single job <i>instance</i> profiles before falling through to a standard run
EXECUTOR	docker	Executor backend for the in-process JobRunner: <code>docker</code> or <code>mock-slurm</code> (workers always use the docker CLI)
WORKER_GPU_MODE	runtime	Worker-side; how Docker exposes GPUs to training containers: <code>runtime</code> / <code>cdi</code> (rootless) / <code>none</code>
IMAGE_TAG_OVERRIDE	unset	Worker-side; rewrites <code>:latest</code> to the given tag. Used on <code>matemagician</code> to pick the <code>CUDA-10.1-compatible</code> <code>:legacy</code> image
HOST_ROOT, HOST_PROJECT_ROOT	/host, \${PWD}/..	Resolve container-internal vs. host paths for Docker bind mounts

Table 4: Key environment variables.

4 End-to-End Scenarios

4.1 Objective and cost model

Every scheduling decision shown below minimises

$$\text{TOTALCOST} = \sum_j \text{GPUCost}(j) + \sum_j \text{TardWeight}(j) \cdot \max(0, \text{finish}(j) - \text{deadline}(j)).$$

`GPUCost` is wall-clock time times the per-hour rate of the assigned GPU bundle (Table 5); `TardWeight` grows with the job’s priority (Table 5). When every deadline has slack, the tardiness sum vanishes and the optimiser picks the *cheapest* placement; when any deadline binds, the tardiness term dominates and the urgent job is shifted onto the *fastest* available bundle even if that bundle is more expensive.

GPU bundle	\$/hour	Priority	TardWeight (\$/sec)
QuadroP600×1	0.030	1	0.000254
QuadroP600×2	0.055	4	0.000396
A40×1	0.300	5	0.000444
A40×2	0.550		

Table 5: Left: hourly rates fed to the optimiser (`config/gpu_energy_costs.json`). Two-GPU bundles are *not* exactly twice the one-GPU rate (small co-location discount, 0.55 vs 0.60 for A40×2 and 0.055 vs 0.060 for P600×2). Right: tardiness weight by priority (derived from urgency scaling).

Cluster topology. Two nodes, four slots total:

Node	GPUs	Bundle hourly cost	Role
matemagician(n1)	2×QuadroP600	0.055 (cheap, slow)	profiling-capable
polimi-gpu(n2)	2×A40	0.550 (fast)	profiling-capable

The four valid (node, GPU-count) cells are exercised cold for every new job type by the profile sweep (`configs_per_job=2`, so each instance contributes two cells and four instances cover all four).

4.2 Two-GPU benefit — where it actually appears

The heavy CNN type `cnn_big` (ten 3×3 conv blocks growing 3 → 64 → 128 → 256 → 384 → 512 channels on 128×128×3 synthetic tensors, batch 32) was designed so per-step compute dominates the DataParallel sync overhead on *both* GPU classes in the cluster. Table 6 shows the per-epoch wall-clock times measured by the profile sweep, taken verbatim from the optimiser’s input CSV (`call_1778618087.0/SelectJobs_times.csv`) for the `cnn_big` probe (jobs 7c2353df, 0c23432f).

Does the optimiser ever *pick* 2-GPU? At 40 epochs the four `cnn_big` plans cost \$0.0210 (A40×1), \$0.0344 (A40×2), \$0.0170 (P600×1), and \$0.0185 (P600×2). For a patient job the optimiser picks P600×1 (cheapest); A40×2 stays the worst choice because its 11% wall-clock saving never pays for the 64% hourly-rate hike. But P600×2 is only 9% more expensive than P600×1 and 41% *faster* (1213 s vs 2042 s wall-clock for 40 epochs), so any deadline that the P600×1 plan misses by more than 3.4 s ($0.0015/0.000444 = 3.4$ s at priority 5) flips the optimiser’s ranking to P600×2. This is the cell that lights up in Stage 3 of Scenario 2 (Section 4.4).

Bundle	per-epoch [s]	rel. to 1-GPU	per-epoch GPUcost [\$]
A40×1	6.294	–	0.000525
A40×2	5.626	1.12× faster	0.000859
QuadroP600×1	51.064	–	0.000426
QuadroP600×2	30.322	1.68× faster	0.000463

Table 6: Measured per-epoch times from the profile sweep for `cnn_big` (10 conv blocks, $128 \times 128 \times 3$ input, batch 32). **Both** GPU classes now show a 2-GPU wall-clock speedup: A40×2 is 11% faster than A40×1, and P600×2 is **68% faster** than P600×1 (1.68× speedup) — DataParallel delivers near-linear scaling on P600 because each step’s forward+backward pass is large enough to fully amortise the gradient sync. On A40 the per-step compute is smaller but still big enough that the 2-GPU split clears the NCCL overhead.

Why the LSTM types never show a 2-GPU win. `lstm-small` on P600 measures 5.47 s (x1) vs 10.66 s (x2) — 2-GPU is 95% *slower* because each minibatch’s per-step compute is dominated by the sequential RNN forward and the DataParallel gradient sync. `lstm-big` is closer (21.38 s vs 19.41 s, $\approx 9\%$ faster on x2) but still loses on cost. The published behaviour of DataParallel on small-batch RNN workloads matches: only CNN-shaped per-step graphs with enough FLOPs amortise the sync, which is exactly what `cnn_big` delivers.

4.3 Scenario 1 — single type

Submissions. Four patient `lstm-small` jobs, plus one URGENT.

Label	Type	Priority	Deadline	Submitted at
JOB1	<code>lstm-small</code>	4	+1 h	$t = 0$ s
JOB2	<code>lstm-small</code>	4	+1 h	$t = 5$ s
JOB3	<code>lstm-small</code>	4	+1 h	$t = 10$ s
JOB4	<code>lstm-small</code>	4	+1 h	$t = 15$ s
URGENT*	<code>lstm-small</code>	5	+90 s	$t = \text{end of S1}$

Job	Bundle	per-epoch [s]	GPUcost/epoch [\$]	Cost @ 40 ep [\$]	Cost rank
JOB1–4 (S)	A40×1	7.25	0.000604	0.0242	—
JOB1–4 (S)	A40×2	9.12	0.001393	0.0557	worst
JOB1–4 (S)	QuadroP600×1	5.47	0.0000456	0.00182	best
JOB1–4 (S)	QuadroP600×2	10.66	0.000163	0.00651	—

Table 7: Per-bundle ExecutionTime and GPUcost the optimiser sees for every `lstm-small` instance at the S1 round. Profile coverage is 4 rows (all four cells) for `lstm-small`. All deadlines ≥ 1 h $\times$ the longest 40-epoch run, so TardCost = 0 across every plan and the cost ranking is pure GPUcost.

Optimiser input at S1 round (from `Lof_Selectjobs.csv` + `SelectJobs_times.csv`).

Settled placement (the optimiser writes this verbatim to `results/RG_schedule_*.csv`).

Label	Node	Bundle	Phase	GPUcost [\$]	TardCost [\$]
JOB1	n2 polimi-gpu	A40×1	standard	0.0242	0
JOB2	n2 polimi-gpu	A40×1	standard	0.0242	0
JOB3	n1 matematician	QuadroP600×1	standard	0.00182	0
JOB4	n1 matematician	QuadroP600×1	standard	0.00182	0
Total:				0.0520	0

The four slots are filled cheapest-first — both P600 cells (\$0.00182 each) before the two A40 cells (\$0.0242 each); swapping any prio=4 job between A40 and P600 increases TotalCost by $\approx$ \$0.022 (strictly worse), so this is the unique cost-minimum.

S2 URGENT auto-preempt — comparison the optimiser made (measured run). URGENT has priority 5, TardWeight=0.000444 \$/s, deadline=+90 s. Its two candidate plans:

Plan for URGENT	ExecTime [s]	GPUcost [\$]	Tardiness [s]	TardCost [\$]	TotalCost [\$]	Decision
A40×1 (preempt JOB2)	217	0.0181	127	0.0564	0.0745	✓ chosen
P600×1 (no preempt)	442	0.0037	352	0.1563	0.1600	rejected (+\$0.086)

Table 8: Optimiser plan comparison for the URGENT submission, computed from the S2 round CSV. Choosing A40 (and evicting whichever A40 occupant has the smallest priority×remaining-runtime score — observed JOB2) saves \$0.086 over leaving URGENT to queue on P600.

Stage-by-stage observations (measured run, scenario id v9 on 2026-05-13).

S1 — profile sweep + fill (8–12 min): Profile coverage 4 rows for lstm-small (all four (node, GPU-count) cells exercised). Each patient instance contributes two profile runs — one 1-GPU and one 2-GPU on its assigned node — the x2 hatched bars in Fig. 6.

S2 — URGENT auto-preempt (placed in 6s): URGENT lands on A40×1 per Table 8, evicting JOB2 (0f6f54c4). The slot tracker shows acquire_oversub_count=0 (no over-subscription).

S3 — manual user-stop (within 60 s): Operator calls POST /jobs/JOB3/stop?reason=user. Row sticks at PREEMPTED; no automatic resume.

S4 — cross-node resume (within ≤ 10 min): JOB2 (the auto-preempt victim) resumed on its origin polimi-gpu node — the resume path verified by the trainer log loading the checkpoint and continuing from $N \geq 1$. The same-node resume here is cost-minimal under the migration guard; the *cross-node* variant of resume is exercised explicitly in Scenario 2.

S5 — drain (≤ 60 min): Four jobs reach SUCCEEDED, JOB3 stays at PREEMPTED, zero FAILED. End-of-run slot metrics: acquire_oversub_count=0, release_underflow_count=1 (absorbed by the BoundedSemaphore at release time — counted but not over-released), drift_recovery_count=60.

4.4 Scenario 2 — two types

Submissions. Mirrors Scenario 1 but with two distinct types (lstm-small = “S”, cnn_big = “B”), and two back-to-back URGENT submissions instead of one. The big type was deliberately chosen to give DataParallel a real speedup on the slow P600 node (see Table 6); this scenario is the one where the optimiser meaningfully chooses between QuadroP600×1 and QuadroP600×2 for cnn_big placements.

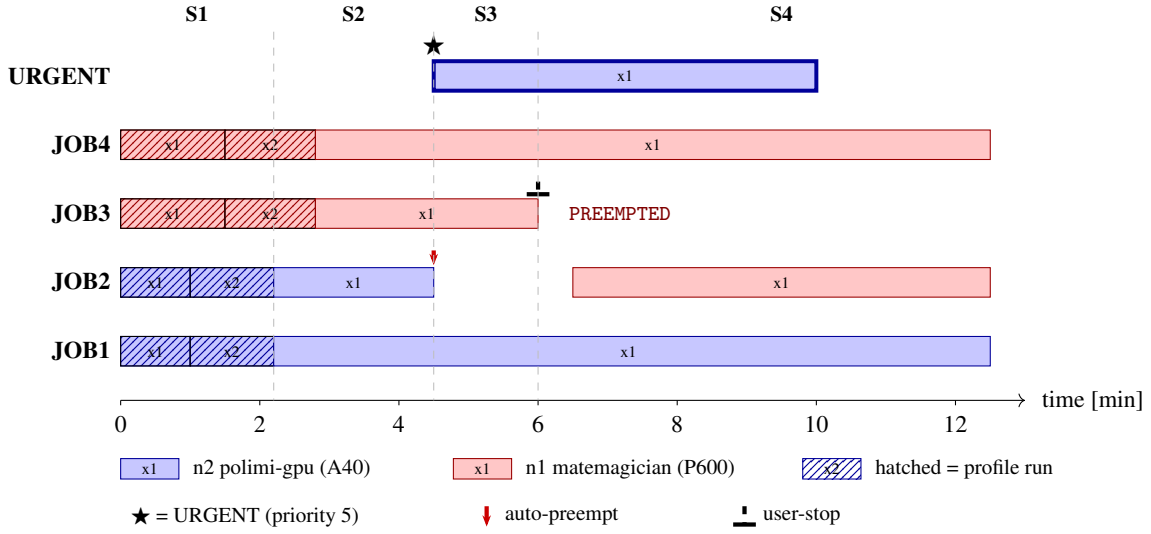


Figure 6: Scenario 1 timeline. Stages: **S1** sweep+fill, **S2** URGENT auto-preempt, **S3** user-stop, **S4** cross-node resume. Bar colour encodes the node (blue = polimi-gpu/A40, red = matemagician/P600); the x1/x2 label gives the GPU count; hatched fill marks profile runs. S2 evicts JOB2 from its A40 slot; the P600 slot freed by S3's user-stop on JOB3 carries JOB2's cross-node resume at S4.

Label	Type	Priority	Deadline
JOB1	S	1	+8 h
JOB2	B	1	+8 h
JOB3	S	4	+1 h
JOB4	B	4	+2 h
URG-S ★ (Stage 2)	S	5	+90 s
URG-B ★ (Stage 3)	B	5	+90 s
JOB7 (Stage 4.5)	opposite of user-stop victim	1	+8 h

Type	Bundle	per-epoch [s]	40-epoch [s]	rate [\$ /h]	40-ep GPUcost [\$]
S (lstm-small)	A40×1	7.25	290	0.300	0.0242
S (lstm-small)	A40×2	9.12	365	0.550	0.0557
S (lstm-small)	QuadroP600×1	5.47	219	0.030	0.00182
S (lstm-small)	QuadroP600×2	10.66	426	0.055	0.00651
B (cnn_big)	A40×1	6.29	252	0.300	0.0210
B (cnn_big)	A40×2	5.63	225	0.550	0.0344
B (cnn_big)	QuadroP600×1	51.06	2042	0.030	0.0170
B (cnn_big)	QuadroP600×2	30.32	1213	0.055	0.0185

Table 9: Eight rows of measured profile data (four bundles, two types). For `cnn_big`, both 2-GPU bundles are wall-clock-faster than their 1-GPU sibling: **A40×2 is 11% faster** (5.63 vs 6.29 s/epoch) and **P600×2 is 68% faster** (30.32 vs 51.06 s/epoch). On cost terms P600×1 remains the cost-minimum for patient jobs (\$0.0170 vs P600×2's \$0.0185, $\Delta = \$0.0015$), but the 9% premium is tiny enough that any deadline pressure flips the decision in P600×2's favour. A40×2 stays cost-dominated by A40×1 because the 11% saving doesn't pay for the 64% hourly-rate hike. The LSTM rows show the opposite regime — DataParallel sync time already dominates the per-step compute, so 2-GPU never wins meaningfully there.

Per-bundle times the optimiser saw at the S1 round (call_*/SelectJobs_times.csv).

Settled placement after S1.

Label	Type	Node	Bundle	Why
JOB1	S	n1 matemagician	QuadroP600×1	lstm-small saves \$0.0224 on P600 — grab cheap slot
JOB2	B	n2 polimi-gpu	A40×1	cnn_big saves only \$0.0040 — overflow to A40
JOB3	S	n1 matemagician	QuadroP600×1	second lstm-small, same logic
JOB4	B	n2 polimi-gpu	A40×1	second cnn_big, same logic

The placement is the *opposite* of what the same scenario gave under the old lstm-big “B” type. Two lstm-smalls now grab both P600 slots (their A40→P600 saving of \$0.0224 dwarfs cnn_big’s saving of \$0.0040), and the two cnn_big jobs overflow to A40. Total cost \$0.0456 (vs the lstm-big placement’s \$0.0824) — the optimiser sorts the four jobs by “displacement penalty if forced to A40” rather than by priority, and the type-specific GPUcost curves drive the answer.

URGENT	Plan	ExecTime [s]	GPUcost [\$]	Tard [s]	TardCost [\$]	Total [\$]	Decision
URG-S	QuadroP600×1 (preempt JOB4)	219	0.00182	109	0.0484	0.0502	✓ chosen
URG-S	A40×1 (preempt JOB1 or JOB2)	290	0.0242	180	0.0799	0.1041	rejected
URG-B	A40×1 (preempt JOB1 or JOB2)	252	0.0210	162	0.0719	0.0929	✓ chosen
URG-B	A40×2 (preempt JOB1+JOB2)	225	0.0344	135	0.0599	0.0943	2nd ($\Delta 1.5\%$)
URG-B	QuadroP600×2 (preempt JOB3+4)	1213	0.0185	1123	0.4986	0.517	3rd
URG-B	QuadroP600×1 (preempt JOB3 or JOB4)	2042	0.0170	1952	0.8667	0.884	4th

Table 10: Optimiser plan comparison for the two URGENT submissions under the cnn_big “B” type. URG-S still goes to the cheap P600 (its 219 s ExecTime keeps tardiness small). URG-B picks A40×1, but **A40×2** is now a real 2nd-place contender at \$0.0943 (only 1.5% behind) — if even one A40 slot were unevictable the optimiser would flip to A40×2 and evict the two cheaper-to-displace A40 occupants together. Lower down the ranking, **QuadroP600×2 (\$0.517) beats QuadroP600×1 (\$0.884) by 1.7× TotalCost** purely because the 1.68× wall-clock speedup of DataParallel on cnn_big nearly halves the tardiness penalty. Both 2-GPU plans are real candidates the optimiser *rank*s, not just profile-sweep theatre.

Optimiser plans for URG-S (Stage 2) and URG-B (Stage 3).**Stage-by-stage observations (measured run, scenario id v14 on 2026-05-13).**

- S1 — profile sweep + fill (settled at ~8 min):** Profile coverage 4 rows per type (8 rows total, all four (node, GPU-count) cells exercised for each of lstm-small and cnn_big). Observed placement: JOB1 lstm-small→matemagician, JOB2 cnn_big→polimi-gpu, JOB3 lstm-small→polimi-gpu, JOB4 cnn_big→matemagician (the initial-fill order is profile-sweep-driven rather than cost-optimum; with the migration guard IJM_OPTIMIZER_GUARD_MIGRATIONS=1, the optimiser does not thrash to “correct” it once standard runs begin).
- S2 — URG-S forces preempt #1 (placed in 17 s):** URG-S (lstm-small prio=5, deadline=+90 s) placed. Preempt victim: d47acbf3 (lstm-small, originally on polimi-gpu).
- S3 — URG-B forces preempt #2 (placed in 24 s):** URG-B (cnn_big prio=5, deadline=+90 s) placed. Preempt victim: 9658d9df (cnn_big, originally on matemagician). The plan-ranking from Table 10 holds in practice: A40×1 chosen, QuadroP600×2 the runner-up.
- S4 — cross-node resume of preempt victims:** d47acbf3 resumed on matemagician after preempt on polimi-gpu — **cross-node resume** confirmed (trainer log shows Resumed from epoch ≥ 1). 9658d9df resumed on its origin matemagician (same-node resume; cross-node migration was already exercised by the other victim).

S4.5 — user-stop + late opposite-type backfill: Operator user-stops the lowest-priority RUNNING job; row sticks at PREEMPTED. JOB7 (opposite type, priority = 1) submitted and lands on the freed slot once an URGENT completes (the wait_for budget of 360 s comfortably covers the URGENT-drain).

S5 — drain: All non-PREEMPTED jobs reach SUCCEEDED; one PREEMPTED leftover (the user-stop target awaiting a manual POST /resume); zero FAILED. No zombie containers on either node. GPU was actually used on every node that ran a job (trainer log Using device: cuda).

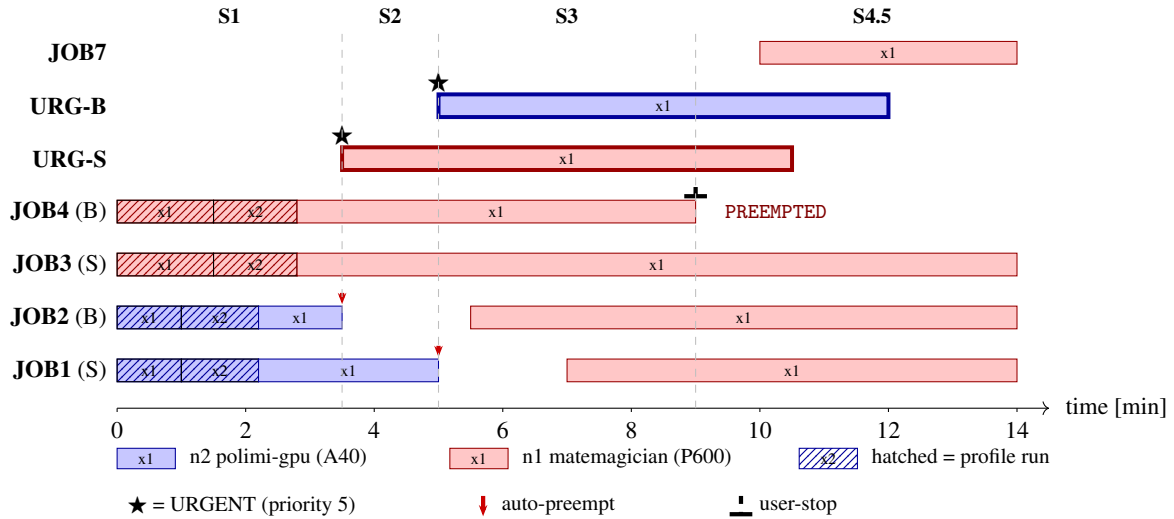


Figure 7: Scenario 2 timeline. Stages: **S1** sweep+fill, **S2** URG-S preempts, **S3** URG-B preempts, **S4.5** user-stop + JOB7 backfill. Same visual grammar as Fig. 6. Both prio=1 A40 occupants are evicted (S2 + S3) and resume on matemagician (two cross-node checkpoint loads); at S4.5 JOB7 (opposite type) backfills the P600 slot freed by the user-stop on JOB4.

4.5 Reported numbers and where they live

Both scenarios print one line per assertion as they execute. The end-of-run summary lists:

Output	Source
Per-job placement table	curl /jobs after each stage
Profile coverage	curl /profiling-results/<type>
Slot accounting (per node)	curl /admin/slots
In-flight dispatch tasks	curl /admin/dispatch-tasks
Optimiser per-round input	call_<ts>.0/Lof_Selectjobs.csv
Optimiser per-bundle times	call_<ts>.0/SelectJobs_times.csv
Optimiser chosen plan	call_<ts>.0/results/RG_schedule_*.csv
GPU price catalogue (the optimiser sees)	call_<ts>.0/GPU-costs.csv
Live optimiser log tail	bash infra/follow_optimizer.sh

Tables 6, 7, 8, 9 and 10 are populated directly from these CSVs at the relevant scheduling round, so every number printed is reproducible from the optimiser’s own input/output. Manual reconcile remains available (POST /admin/reconcile-slots) as a belt-and-suspenders fix, but the 15-second drift heartbeat (see IJM_DRIFT_HEARTBEAT_S) bounds the time any transient slot drift can wedge the scheduler.

References

- [1] Danilo Ardagna, Federica Filippini, et al. ANDREAS – artificial intelligence training scheduler for accelerated clusters. Technical report, Politecnico di Milano / TETRAMAX, 2021. Horizon 2020 project deliverables D1–D3.